Entitlements & Distribution

Access control, digital delivery, license lifecycle and affiliate commissions

1. Entitlements

HLD / LLD

Entitlements is the authoritative access decision service. It receives trusted events from billing/payments/distribution, records grants and revocations, and exposes a low-latency check API to CPG products.

- Grant source must be traceable: order line, subscription period, credit plan, manual grant or approved promotion.
- Use a unique grant source key to make repeated events idempotent.
- Check status, start time, expiry and feature/product scope. Define precedence when multiple grants exist.
- Payment reversal/refund behavior is policy-driven by product type; do not globally revoke all access on any partial refund.
- Use an event history or append-only entitlement changes for audit. Current-state projection can be materialized.
- Downstream applications should fail closed for protected premium access when the authoritative check cannot be established, unless a documented cache policy says otherwise.

Schema and DTO

```
type EntitlementDoc = {
  _id:ObjectId; tenantId:ObjectId; publicId:string; customerId:ObjectId;
  productId?:ObjectId; featureKey?:string; sourceType:string; sourceId:string;
  status:"active"|"suspended"|"revoked"|"expired";
  startsAt:Date; expiresAt?:Date; createdAt:Date; updatedAt:Date;
};
// Index {tenantId, customerId, status, expiresAt}
// Unique source key on {tenantId, sourceType, sourceId, featureKey}

export type CheckEntitlementDTO = { customerId:string; productId?:string; featureKey?:string };
export type EntitlementCheckDTO = { allowed:boolean; reason:string; expiresAt:string|null };
```

Endpoints and UI

Route	Contract
POST /api/v1/entitlements/check	200 EntitlementCheckDTO; authenticate calling app and authorize customer scope.
GET /api/v1/entitlements	List customer entitlements, cursor-paginated.
POST /api/v1/entitlements/:id/revoke	Privileged/audited; requires reason and expectedVersion.
POST /api/v1/entitlements/:id/suspend	Privileged/audited; supports later resume if policy permits.

UI: Customer -> Entitlements tab -> active/expired/revoked grants -> source purchase/subscription -> audit timeline. Manual override requires explicit reason and confirmation. Avoid displaying internal source IDs to unauthorized roles.

Sequence

```
sequenceDiagram
  participant Pay as Billing Event Worker
  participant E as EntitlementService
  participant DB as MongoDB
  participant App as CPG Product
  participant SDK as kubera.entitlements
  Pay->>E: grant(sourceId, customer, product)
  E->>DB: Upsert idempotently by source key
  E-->>Pay: Grant result
  App->>SDK: entitlements.check(dto)
  SDK->>E: POST /entitlements/check
  E->>DB: Query active grants by tenant/customer/feature
  E-->>SDK: EntitlementCheckDTO
  SDK-->>App: allowed/reason/expiry
```

2. Storefront and delivery

HLD / LLD

Distribution owns public product presentation, checkout handoff and post-purchase delivery. Files are stored in private object storage. Distribution must be idempotent and independent from payment retry behavior.

- Only published products and active prices are visible on storefront.
- Checkout session creation is delegated to Checkout; storefront never calculates authoritative prices.
- Fulfillment task has a stable source key (order line + fulfillment type). Retry reuses task and does not charge again.
- Private assets use short-lived signed URLs; authorization is checked before issuing them.
- Email notification is a side effect of delivery state, not proof of delivery itself.
- Use a transactional outbox after payment/order confirmation; retry failures with backoff and dead-letter visibility.

Schema and DTO

```
type DeliveryDoc = {
  _id:ObjectId; tenantId:ObjectId; publicId:string; orderId:ObjectId;
  orderLineRef:string; customerId:ObjectId; type:"download"|"license"|"access"|"custom";
  status:"queued"|"processing"|"delivered"|"failed"|"cancelled";
  attempts:number; lastError?:string; deliveredAt?:Date; createdAt:Date; updatedAt:Date;
};
// Unique {tenantId, orderLineRef, type}; index {tenantId, status, updatedAt}

export type DeliveryDTO = {
  id:string; orderId:string; type:string; status:string; attempts:number;
  deliveredAt?:string; lastErrorCode?:string;
};
```

Endpoints

Route	Contract
GET /api/v1/storefront/:slug	Public published catalog DTO only; 404 for unpublished/unknown slug.
POST /api/v1/deliveries/:id/retry	Operator or authorized customer workflow; policy checks; 202 queued.
GET /api/v1/deliveries/:id	Authorized customer/tenant; never reveal private asset storage credentials.
POST /api/v1/assets	Signed upload initiation for tenant admin; validates content type and max size.
GET /api/v1/deliveries/:id/download	Reauthorize entitlement/order and issue short-lived signed URL; no direct public object path.

UI

Screen	Components / state
Storefront product page	ProductHero, PriceSelector, CheckoutButton; public query cache; checkout session created server-side.
Checkout handoff	Redirect/embedded checkout boundary; CSP and allowed origins configured.
Purchase success	OrderStatus, DeliveryStatus, Retry/Support CTA; poll server state, handle delayed webhook.
Asset management	AssetTable, UploadDialog, VersionStatus; upload progress local, asset metadata server state.
Delivery operations	DeliveryQueue, AttemptTimeline, RetryButton; permission-guarded retry.

Sequence

```
sequenceDiagram
  participant UI as Storefront
```

```
  participant SDK as kubera.distribution
  participant API as Express
  participant DB as MongoDB
  participant PSP as Payment Provider
  participant W as Fulfillment Worker
  participant OBJ as Object Storage
  UI->>SDK: checkout.createSession(...)
  SDK->>API: POST /checkout/sessions
  API-->>UI: checkoutUrl
  PSP->>API: Verified payment webhook
  API->>DB: Payment state + outbox event
  W->>DB: Create/claim delivery by stable source key
  W->>OBJ: Verify asset or create signed download flow
  W->>DB: Mark delivery delivered/failed
  UI->>SDK: deliveries.get(id)
  SDK->>API: GET /deliveries/:id
  API-->>UI: DeliveryDTO
```

3. License management

Rules and schema

- Generate high-entropy license secrets with a cryptographically secure random generator.
- Persist a hash for verification when the original key need not be recovered; show the plaintext only at issuance.
- Activation is atomic and enforces maxActivations. Use a separate licenseActivations collection with unique device/activation identity scoped to license.
- Define whether deactivation frees a slot, whether devices can be reset, and whether licenses are node-locked or user-bound.
- License status follows purchase/refund/expiry policy, but manual override requires audit.

```
type LicenseDoc = {
  _id:ObjectId; tenantId:ObjectId; publicId:string; productId:ObjectId; customerId:ObjectId;
  keyHash:string; status:"issued"|"active"|"suspended"|"revoked"|"expired";
  maxActivations:number; activationCount:number; expiresAt?:Date;
  sourceOrderId:ObjectId; createdAt:Date; updatedAt:Date;
};
type LicenseActivationDoc = {
  tenantId:ObjectId; licenseId:ObjectId; activationId:string;
  deviceFingerprintHash?:string; activatedAt:Date; deactivatedAt?:Date;
};
// Unique {tenantId, keyHash}; unique {tenantId, licenseId, activationId}
export type ActivateLicenseDTO = { licenseKey:string; activationId:string; deviceFingerprint?:string };
export type LicenseActivationDTO = { licenseId:string; status:string; activatedAt:string; expiresAt?:string };
```

API and UI

Route	Contract
POST /api/v1/licenses/activate	Public or app-authenticated with rate limiting; 200 activation result; generic invalid-key response.
POST /api/v1/licenses/:id/deactivate	Authorized license owner/app; idempotent.
GET /api/v1/licenses	Tenant/admin or authorized customer list; never return stored hash.
POST /api/v1/licenses/:id/revoke	Privileged, reason required, audit event.

UI: License issuance is part of fulfillment; customer sees key once with copy/download action. License detail shows masked key, activations, expiry and status. Admin revoke requires confirmation and reason. Rate-limit activation and avoid revealing whether a particular key belongs to a specific customer.

Sequence

```
sequenceDiagram
  participant App as Customer App
  participant SDK as kubera.licenses
  participant API as Express
  participant L as LicenseService
  participant DB as MongoDB
  App->>SDK: licenses.activate(dto)
  SDK->>API: POST /licenses/activate
  API->>L: validate key + tenant/product policy
  L->>DB: Find hash; atomically reserve activation slot
  alt valid and capacity available
    L->>DB: Insert activation
```

```
  API-->>App: 200 LicenseActivationDTO
else invalid or full
  API-->>App: 422 LICENSE_INVALID_OR_LIMIT_REACHED
end
```

4. Affiliate attribution and commissions

LLD

- Store attribution under tenant and campaign policy: first-click/last-click, attribution window, cookie/session reference and consent requirements.
- Do not trust affiliateCode from the browser as proof of attribution; resolve it server-side and validate campaign status.
- Conversion is linked to a paid order only once; unique conversion source key prevents duplicate commissions.
- Commission basis is explicitly defined (gross, net of discounts, excluding tax, excluding refunded amount). Version the commission policy used by each conversion.
- Refund/chargeback creates a commission adjustment/reversal; never delete the original commission record.
- Initial release calculates and reports liabilities. Automated payouts are a separate phase with payee verification, tax and compliance requirements.

Schema / DTO

```
type AffiliateConversionDoc = {
  _id:ObjectId; tenantId:ObjectId; affiliateId:ObjectId; orderId:ObjectId;
  attributionRef:string; policyVersion:string; status:"pending"|"eligible"|"reversed";
  eligibleAmountMinor:number; currency:string; createdAt:Date;
};
type CommissionDoc = {
  _id:ObjectId; tenantId:ObjectId; affiliateId:ObjectId; conversionId:ObjectId;
  basisAmountMinor:number; commissionMinor:number; currency:string;
  status:"pending"|"approved"|"payable"|"reversed"; createdAt:Date;
};
// Unique {tenantId, orderId, affiliateId} or configured conversion key.
// Unique commission source reference.
export type CreateAffiliateDTO = { name:string; code:string; commissionModel:"percentage"|"fixed"; rateBps?:number; fixedAmountMino
export type AffiliateReportDTO = { affiliateId:string; clicks:number; conversions:number;
  eligibleRevenueMinor:number; commissionMinor:number; currency:string; periodStart:string; periodEnd:string };
```

API and UI

Route	Contract
POST /api/v1/affiliates	Tenant admin creates affiliate/campaign; 201.
GET /api/v1/affiliates	Admin list, cursor-paginated.
GET /api/v1/affiliate/reports	Period and affiliate filters; aggregate report DTO.
GET /api/v1/affiliates/:id/commissions	Commission ledger with status and adjustments.
POST /api/v1/commissions/:id/approve	Privileged action; reason/audit; no payout in MVP.

UI: Affiliate list -> campaign detail -> attribution settings -> performance report -> conversion detail -> commission adjustments. Charts are server state; report filters live in URL; approval actions require role and confirmation.

Sequence

```
sequenceDiagram
  participant V as Visitor
  participant S as Storefront
  participant A as AttributionService
  participant DB as MongoDB
  participant W as Order Worker
  participant C as CommissionService
  V->>S: Open affiliate link
  S->>A: Resolve code and record eligible attribution
  A->>DB: Save attribution reference
  V->>S: Complete checkout
  W->>DB: Observe order.paid event
  W->>C: Create conversion idempotently
  C->>DB: Save conversion + commission liability
  C->>DB: Later refund event -> adjustment/reversal
```